Strings e arquivos em C

Strings e arquivos em C

Material didático de apoio — Linguagem C

Sumário

- 1. Strings
  - 1.1. O que é uma string
  - 1.2. Declaração e inicialização
  - 1.3. Entrada e saída de strings
  - 1.4. Acessando e modificando caracteres
  - 1.5. Funções da biblioteca string.h
  - 1.6. Vetores de strings
  - 1.7. Exemplo: iniciais maiúsculas
- 2. Arquivos
  - 2.1. Por que usar arquivos
  - 2.2. Arquivos texto e arquivos binários
  - 2.3. O ponteiro FILE *
  - 2.4. Abertura e fechamento de arquivos
  - 2.5. Escrita em arquivos texto
  - 2.6. Leitura de arquivos texto
  - 2.7. Arquivos binários
  - 2.8. Acesso direto com fseek, ftell e rewind
- 3. Boas práticas e erros comuns

Programas costumam lidar com texto: nomes, mensagens, caminhos de arquivo, linhas de um relatório. Em C, esse texto é representado por strings. Quando é preciso guardar informações depois que o programa termina, entram os arquivos. Este material apresenta as duas ideias em conjunto: primeiro o funcionamento das strings e as funções de manipulação; depois a abertura, escrita, leitura e navegação em arquivos texto e binários.

1. Strings

1.1. O que é uma string

Uma string é uma sequência de caracteres armazenada em memória. A linguagem C não oferece um tipo nativo para strings. Em vez disso, usa uma convenção: os caracteres ficam em posições consecutivas de um vetor e a sequência termina com o caractere nulo, escrito `'\0'`, cujo valor numérico é zero.

Essa convenção é o que permite que funções como `printf` e `strlen` saibam onde o texto acaba. Elas avançam pela memória até encontrar o terminador. Se ele não existir, a leitura continua por regiões que não pertencem ao vetor, produzindo resultados imprevisíveis.

A diferença entre aspas simples e duplas é importante:

- `'A'` representa um único caractere;
- `"A"` representa uma string com dois bytes: o caractere `A` e o terminador `'\0'`.

Considere:

```
char palavra[] = "casa";
```

A memória fica assim:

Índice	0	1	2	3	4
Conteúdo	`c`	`a`	`s`	`a`	`'\0'`

O texto visível tem quatro letras, mas o vetor ocupa cinco bytes. Essa diferença entre quantidade de caracteres e espaço reservado aparece em quase todos os erros envolvendo strings.

1.2. Declaração e inicialização

Uma string pode ser declarada como um vetor de `char`. O tamanho precisa comportar todos os caracteres e mais o terminador:




```
char cidade[13];
```

Esse vetor pode guardar até 12 caracteres úteis. Para armazenar `"São Paulo"` (9 caracteres), o tamanho 13 é suficiente, mas sobra espaço. Já `char cidade[9]` seria pequeno demais: faltaria lugar para o `'\0'`.

Há três maneiras comuns de iniciar uma string:

```
/* Lista explícita de caracteres */
char a[6] = {'c', 'a', 's', 'a', 's', '\0'};

/* Literal com tamanho fixo */
char b[6] = "casas";

/* Tamanho calculado pelo compilador */
char c[] = "casas";
```

Nas duas últimas formas, o compilador acrescenta o terminador automaticamente. A diferença é que `b` reserva exatamente seis bytes, enquanto `c` reserva o espaço necessário para o literal.

Se a string precisar crescer depois, reserve uma folga:

```
char texto[50] = "casa";
char justo[]   = "casa";
```

`texto` aceita novos caracteres até o limite de 49. `justo` ocupa apenas cinco bytes e não comporta acréscimos.

Também é possível usar ponteiros:

```
const char *mensagem = "bom dia";
```

Nesse caso, `mensagem` aponta para um literal que não deve ser alterado. Para modificar o conteúdo, use um vetor:

```
char mensagem[] = "bom dia";
mensagem[0] = 'B';
```

Regra prática: use `char vetor[TAMANHO]` quando o texto será alterado; use `const char *` quando apenas será lido.

1.3. Entrada e saída de strings

A saída pode ser feita com `printf` e o formato `%s`:

```
#include <stdio.h>

int main(void) {
    char nome[] = "Marina";

    printf("Nome: %s\n", nome);
    return 0;
}
```



A função `puts` é uma alternativa mais curta: ela imprime a string e acrescenta uma quebra de linha.

```
puts("Nome: Marina");
```

Para entrada, `scanf` com `%s` é limitado: ele para no primeiro espaço. Além disso, sem uma largura máxima, pode ultrapassar o tamanho do vetor:

```
char nome[30];
scanf("%29s", nome); /* lê no máximo 29 caracteres */
```

Para ler uma linha inteira, use `fgets`:

```
#include <stdio.h>
#include <string.h>

int main(void) {
    char linha[100];

    printf("Digite uma frase: ");
    if (fgets(linha, sizeof linha, stdin) == NULL) {
        return 1;
    }

    size_t n = strlen(linha);
    if (n > 0 && linha[n - 1] == '\n') {
        linha[n - 1] = '\0';
    }
```

```
    printf("Você digitou: %s\n", linha);
    return 0;
}
```

`fgets` recebe o vetor, o tamanho máximo e o fluxo de entrada. Ela lê até `n - 1` caracteres, para no `'\n'` ou no fim da entrada e sempre termina a string com `'\0'`. Como a quebra de linha pode fazer parte do resultado, o exemplo a remove manualmente.

Se a linha não couber no vetor, `fgets` lê apenas a parte inicial. O restante continua no fluxo e será lido na próxima chamada. Em programas que precisam tratar linhas longas, vale verificar se o último caractere lido foi `'\n'`.

Não use `gets`. Ela não recebe o tamanho do destino, não verifica limites e foi removida do padrão C11. O substituto adequado é `fgets`.

1.4. Acessando e modificando caracteres

Cada posição da string pode ser lida ou alterada pelo índice, como em qualquer vetor:

```
#include <stdio.h>

int main(void) {
    char palavra[] = "computador";
    char letra = palavra[3];

    printf("%c\n", letra); /* p */
    palavra[0] = 'C';
    printf("%s\n", palavra); /* Computador */

    return 0;
}
```



O índice começa em zero. O último caractere útil de `"computador"` está na posição 9; a posição 10 contém `'\0'`.

Para acrescentar um caractere, é preciso localizar o terminador, verificar se há espaço e escrever o novo caractere seguido de um novo terminador:

```
#include <stdio.h>

int main(void) {
    char texto[10] = "abc"; /* 3 letras + '\0' + folga */
    int i = 0;

    while (texto[i] != '\0') {
        i++;
    }

    if (i < 9) {
        texto[i] = '!';
        texto[i + 1] = '\0';
    }

    printf("%s\n", texto);
    return 0;
}
```

Sem a verificação `i < 9`, o programa poderia escrever fora dos limites do vetor. Esse cuidado é essencial: o compilador não impede acessos inválidos em tempo de execução.

1.5. Funções da biblioteca `string.h`

A biblioteca `string.h` reúne funções para medir, copiar, comparar e localizar strings. Todas dependem de um terminador `'\0'` válido.

`strlen`: medir o comprimento

`strlen(s)` conta os caracteres até o terminador, sem incluí-lo:

```
#include <stdio.h>
#include <string.h>

int main(void) {
    char s[] = "linguagem";

    printf("%zu\n", strlen(s)); /* 9 */
    printf("%zu\n", sizeof s);  /* 10 */
    return 0;
}
```

`strlen` percorre a string em tempo de execução e não conta o `'\0'`. `sizeof` é resolvido em tempo de compilação e mede o vetor inteiro, incluindo o terminador. Em um ponteiro `char *`, `sizeof` devolve o tamanho do ponteiro, não o comprimento do texto.

`strcpy` e `strncpy`: copiar



`strcpy(destino, origem)` copia a string inteira, incluindo o `'\0'`. O destino precisa ter capacidade suficiente:

```
#include <stdio.h>
#include <string.h>

int main(void) {
    char origem[] = "casa nova";
    char destino[20];

    strcpy(destino, origem);
    printf("%s\n", destino);
    return 0;
}
```

`strncpy(destino, origem, n)` limita a cópia a `n` caracteres. Se a origem tiver `n` caracteres ou mais, o resultado pode não terminar com `'\0'`. Para garantir o terminador:

```
strncpy(destino, origem, sizeof destino - 1);
destino[sizeof destino - 1] = '\0';
```

`strcat` e `strncat`: concatenar

`strcat(destino, origem)` acrescenta o conteúdo de `origem` ao final de `destino`. O destino precisa comportar as duas strings e o terminador:

```c
#include <stdio.h>
#include <string.h>

int main(void) {
    char saudacao[30] = "bom";
    char complemento[] = " dia";

    strcat(saudacao, complemento);
    printf("%s\n", saudacao); /* bom dia */
    return 0;
}
```

`strncat(destino, origem, n)` acrescenta no máximo `n` caracteres e sempre termina o resultado. Ainda assim, é necessário reservar espaço para o texto completo.

`strcmp` e `strncmp`: comparar

`strcmp(a, b)` percorre as duas strings comparando os caracteres pela tabela ASCII. O retorno é zero quando são iguais, negativo quando `a` vem antes de `b` e positivo quando `a` vem depois.

```c
#include <stdio.h>
#include <string.h>

int main(void) {
    char a[] = "casa";
    char b[] = "casa";
    char c[] = "Casa";

    printf("%d\n", strcmp(a, b)); /* 0 */
    printf("%d\n", strcmp(a, c)); /* positivo: 'c' > 'C' */
    return 0;
}
```

A comparação diferencia maiúsculas de minúsculas. Para comparar apenas os primeiros `n` caracteres, use `strncmp(a, b, n)`.

Atenção: `a == b` compara endereços, não o conteúdo das strings. Para verificar igualdade textual, use `strcmp`.

`strchr`, `strrchr` e `strstr`: localizar

`strchr(s, c)` devolve um ponteiro para a primeira ocorrência de `c`; `strrchr` devolve a última. `strstr(s, trecho)` procura uma substring. Todas retornam `NULL` quando não encontram nada.

```c
#include <stdio.h>
#include <string.h>

int main(void) {
    char frase[] = "casa, praia, campo";
    char *p = strchr(frase, ',');

    if (p != NULL) {
        printf("Vírgula na posição %ld\n", (long)(p - frase));
    }

    char *q = strstr(frase, "praia");
    if (q != NULL) {
        printf("Trecho encontrado na posição %ld\n", (long)(q - frase));
    }
```

```
    return 0;
}
```

`strtok`: separar campos

`strtok` divide uma string usando um ou mais delimitadores. A primeira chamada recebe a string; as seguintes recebem `NULL`:

```
#include <stdio.h>
#include <string.h>

int main(void) {
    char lista[] = "casa,praia,campo";
    char *item = strtok(lista, ",");

    while (item != NULL) {
        printf("%s\n", item);
        item = strtok(NULL, ",");
    }

    return 0;
}
```

`strtok` altera a string original, trocando os delimitadores por `'\0'`. Por isso, não deve ser usada em literais ou em strings `const`. A função também mantém estado interno e não é adequada para uso simultâneo em várias threads; nesse caso, existe `strtok_r`.

1.6. Vetores de strings

Um vetor de strings pode ser representado como uma matriz de `char`:

```
char cidades[4][30];
```

Cada linha é uma string independente. Para ler várias linhas:

```
#include <stdio.h>
#include <string.h>

int main(void) {
    char cidades[4][30];
    int i;

    for (i = 0; i < 4; i++) {
        printf("Cidade %d: ", i + 1);
        if (fgets(cidades[i], 30, stdin) == NULL) {
            cidades[i][0] = '\0';
            continue;
        }

        size_t n = strlen(cidades[i]);
        if (n > 0 && cidades[i][n - 1] == '\n') {
            cidades[i][n - 1] = '\0';
        }
    }

    for (i = 0; i < 4; i++) {
        printf("%d: %s\n", i + 1, cidades[i]);
    }

    return 0;
}
```

A matriz também pode ser inicializada diretamente:

```
char cidades[3][30] = {
    "São Paulo",
    "Campinas",
    "Santos"
};
```

Quando as strings têm tamanhos muito diferentes, um vetor de ponteiros pode economizar espaço:

```
const char *estados[] = {"SP", "RJ", "MG"};
```

Nesse formato, cada posição guarda um endereço para um literal. O acesso é o mesmo, mas o conteúdo apontado não deve ser modificado.

1.7. Exemplo: iniciais maiúsculas

O programa a seguir lê uma frase e transforma em maiúscula a primeira letra de cada palavra:

```
#include <stdio.h>
#include <string.h>
#include <ctype.h>

int main(void) {
    char frase[100];
    int i = 0;
    int inicio = 1;

    printf("Frase: ");
    if (fgets(frase, sizeof frase, stdin) == NULL) {
        return 1;
    }

    while (frase[i] != '\0') {
        unsigned char c = (unsigned char) frase[i];

        if (inicio && isalpha(c)) {
            frase[i] = (char) toupper(c);
        }

        inicio = isspace(c);
        i++;
    }

    printf("%s", frase);
    return 0;
}
```

A variável `inicio` indica se a posição atual começa uma palavra. Ela vale 1 no início e passa a valer 1 sempre que o caractere anterior é um espaço. O teste `isalpha` evita converter números e sinais de pontuação.

A biblioteca `ctype.h` oferece funções de classificação e conversão:

Função	Uso
`isalpha(c)`	verifica se `c` é letra
`isdigit(c)`	verifica se `c` é dígito

Função	Uso
`isspace(c)`	verifica se `c` é espaço em branco
`isupper(c)` / `islower(c)`	verifica maiúscula / minúscula
`toupper(c)`	converte para maiúscula
`tolower(c)`	converte para minúscula

Essas funções esperam um valor que caiba em `unsigned char` ou o valor `EOF`. Por isso, é recomendável converter o argumento: `toupper((unsigned char) c)`.

2. Arquivos

2.1. Por que usar arquivos

Dados guardados em variáveis existem apenas enquanto o programa está em execução. Ao encerrar, a memória é liberada e as informações se perdem. Arquivos resolvem esse problema: eles mantêm os dados em disco e permitem que sejam consultados em outra execução, por outro programa ou em outro momento.

Arquivos também são úteis quando o volume de dados é grande demais para caber na memória ou quando várias aplicações precisam compartilhar informações. A biblioteca padrão de C oferece uma interface comum para esse acesso, escondendo detalhes do sistema de arquivos.

2.2. Arquivos texto e arquivos binários



Os arquivos podem ser classificados em dois grupos:

Tipo	Conteúdo	Exemplos
Texto	caracteres legíveis, geralmente organizados em linhas	`.txt`, `.csv`, código-fonte
Binário	bytes no formato definido pelo programa	imagens, executáveis, compactados

Em um arquivo texto, as funções de leitura e escrita interpretam quebras de linha e caracteres. Em um arquivo binário, os bytes são gravados e lidos sem essa interpretação. O significado depende inteiramente do formato usado por quem criou o arquivo.

2.3. O ponteiro `FILE *`

O acesso a um arquivo é feito por um ponteiro para `FILE`, tipo declarado em `stdio.h`:

```
#include <stdio.h>

int main(void) {
    FILE *arq;
    /* ... */
    return 0;
}
```

`FILE` representa um fluxo de dados e guarda informações internas sobre o arquivo aberto. O programador não deve acessar seus campos diretamente. Tudo é feito pelas funções da biblioteca.

2.4. Abertura e fechamento de arquivos

A abertura é feita com `fopen`, que recebe o nome do arquivo e o modo:

```
FILE *arq = fopen("contatos.txt", "r");
```

Os modos principais são:

Modo	Comportamento
`"r"`	leitura; o arquivo precisa existir
`"w"`	escrita; cria o arquivo ou apaga o conteúdo existente
`"a"`	escrita no final; cria o arquivo se necessário
`"r+"`	leitura e escrita; o arquivo precisa existir
`"w+"`	leitura e escrita; cria ou apaga o conteúdo existente
`"a+"`	leitura e escrita no final; cria o arquivo se necessário
`"rb"`, `"wb"`, `"ab"`	versões binárias dos modos de leitura, escrita e anexação
`"r+b"`, `"w+b"`, `"a+b"`	leitura e escrita em modo binário

A letra `b` indica modo binário. Em sistemas Unix, ela costuma ter pouco efeito prático; no Windows, evita a conversão automática de quebras de linha.



Depois do uso, o arquivo deve ser fechado:

```
fclose(arq);
```

O fechamento grava os dados que ainda estão em buffer e libera os recursos. Se o programa terminar sem fechar, o sistema pode fechar o arquivo, mas parte do conteúdo pode não chegar ao disco.

`fopen` devolve `NULL` quando a abertura falha. Isso acontece, por exemplo, quando o arquivo não existe, o caminho está errado ou não há permissão de acesso. Verifique sempre:

```
#include <stdio.h>
#include <stdlib.h>

int main(void) {
    FILE *arq = fopen("contatos.txt", "r");

    if (arq == NULL) {
        perror("Erro ao abrir contatos.txt");
        return EXIT_FAILURE;
    }

    /* ... */

    if (fclose(arq) != 0) {
        perror("Erro ao fechar contatos.txt");
        return EXIT_FAILURE;
```

```
    }

    return EXIT_SUCCESS;
}
```

`perror` imprime a mensagem informada seguida da descrição do erro fornecida pelo sistema. É uma maneira simples de obter diagnósticos mais claros.

2.5. Escrita em arquivos texto

`fprintf`

`fprintf` é semelhante a `printf`, mas recebe o arquivo como primeiro argumento:

```
fprintf(arq, "Nome: %s\n", nome);
```

Exemplo de gravação de três linhas:

```
#include <stdio.h>
#include <stdlib.h>

int main(void) {
    FILE *arq = fopen("saida.txt", "w");

    if (arq == NULL) {
        perror("fopen");
        return EXIT_FAILURE;
    }

    fprintf(arq, "Nome: %s\n", "Marina");
    fprintf(arq, "Idade: %d\n", 21);
    fprintf(arq, "Nota: %.1f\n", 9.2);

    fclose(arq);
    return EXIT_SUCCESS;
}
```



`fputs`

`fputs` grava uma string sem acrescentar quebra de linha:

```
fputs("uma linha qualquer\n", arq);
```

O retorno é `EOF` em caso de erro. Para verificar:

```
if (fputs("linha\n", arq) == EOF) {
    perror("fputs");
}
```

`fputc`

`fputc` grava um único caractere:

```
fputc('A', arq);
```

Ele é útil quando o conteúdo é produzido caractere a caractere:

```c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

int main(void) {
    FILE *arq = fopen("texto.txt", "w");
    char frase[100];

    if (arq == NULL) {
        perror("fopen");
        return EXIT_FAILURE;
    }

    printf("Frase: ");
    if (fgets(frase, sizeof frase, stdin) == NULL) {
        fclose(arq);
        return EXIT_FAILURE;
    }

    for (size_t i = 0; i < strlen(frase); i++) {
        fputc(frase[i], arq);
    }

    fclose(arq);
    return EXIT_SUCCESS;
}
```

Em resumo: `fprintf` serve para dados formatados, `fputs` para strings e `fputc` para caracteres isolados.

2.6. Leitura de arquivos texto

A leitura de um arquivo texto pode ser feita caractere a caractere, linha a linha ou por campos formatados. A escolha depende de como o conteúdo foi gravado.

`fgetc`: caractere por caractere

`fgetc(arq)` devolve o próximo caractere como `int` ou `EOF` quando não há mais dados ou ocorre erro. O retorno deve ser guardado em `int`, porque `EOF` é negativo e pode não caber em `char`:

```c
int c;
while ((c = fgetc(arq)) != EOF) {
    putchar(c);
}
```

Esse trecho é útil para copiar ou inspecionar o conteúdo sem assumir nada sobre a existência de linhas.

`fgets`: linha por linha

`fgets(linha, tamanho, arq)` lê uma linha, limitada a `tamanho - 1` caracteres, e termina com `'\0'`:

```c
char linha[128];
while (fgets(linha, sizeof linha, arq) != NULL) {
    printf("%s", linha);
}
```

Se a linha for maior que o vetor, ela será dividida em mais de uma leitura. O `'\n'` também é armazenado quando existe espaço.

`fscanf`: campos formatados

`fscanf` lê valores de acordo com um formato e devolve quantas conversões conseguiu realizar:

```
char nome[40];
int idade;
while (fscanf(arq, "%39s %d", nome, &idade) == 2) {
    printf("%s tem %d anos\n", nome, idade);
}
```

Compare o retorno com a quantidade de campos esperada. Evite `%s` sem largura, pois ele não conhece o tamanho do vetor.

Fim de arquivo e erros

Não use `while (!feof(arq))` para controlar a leitura. A função `feof` só informa que o fim foi alcançado depois de uma tentativa de leitura, o que costuma processar o último item duas vezes. Prefira testar o retorno de `fgetc`, `fgets` ou `fscanf`.

Depois que o laço termina, `ferror(arq)` permite verificar se houve falha de leitura:

```
if (ferror(arq)) {
    perror("Erro de leitura");
}
```

Dessa forma, fim normal e erro são tratados de maneira diferente.

2.7. Arquivos binários



Arquivos binários são adequados para registros de tamanho fixo, grandes volumes de dados e situações em que a representação textual seria lenta ou desperdiçaria espaço. As funções centrais são:

```
size_t fwrite(const void *dados, size_t tamanho, size_t quantidade, FILE *arq);
size_t fread(void *dados, size_t tamanho, size_t quantidade, FILE *arq);
```

`fwrite` grava `quantidade` elementos de `tamanho` bytes. `fread` lê esses elementos. O retorno informa quantos elementos foram efetivamente processados.

Considere uma estrutura de contato:

```
typedef struct {
    char nome[40];
    char telefone[16];
    int idade;
} Contato;
```

Para gravar:

```
#include <stdio.h>
#include <stdlib.h>

typedef struct {
    char nome[40];
    char telefone[16];
```

```
    int idade;
} Contato;

int main(void) {
    Contato contatos[2] = {
        {"Marina", "9999-1111", 21},
        {"Rafael", "9888-2222", 34}
    };

    FILE *arq = fopen("contatos.bin", "wb");

    if (arq == NULL) {
        perror("fopen");
        return EXIT_FAILURE;
    }

    if (fwrite(contatos, sizeof(Contato), 2, arq) != 2) {
        perror("fwrite");
        fclose(arq);
        return EXIT_FAILURE;
    }

    fclose(arq);
    return EXIT_SUCCESS;
}
```

Para ler:

```
#include <stdio.h>
#include <stdlib.h>

typedef struct {
    char nome[40];
    char telefone[16];
    int idade;
} Contato;

int main(void) {
    Contato c;
    FILE *arq = fopen("contatos.bin", "rb");

    if (arq == NULL) {
        perror("fopen");
        return EXIT_FAILURE;
    }

    while (fread(&c, sizeof c, 1, arq) == 1) {
        printf("%-10s %-12s %d\n", c.nome, c.telefone, c.idade);
    }

    fclose(arq);
    return EXIT_SUCCESS;
}
```

Alguns cuidados importantes:

- não grave ponteiros, pois um endereço de memória não faz sentido fora da execução atual;
- estruturas podem conter bytes de preenchimento (*padding*) e o layout pode variar entre plataformas;
- a ordem dos bytes pode ser diferente entre arquiteturas;
- para dados portáteis, prefira gravar campo a campo ou usar um formato textual;
- sempre verifique o retorno de `fread` e `fwrite`.

2.8. Acesso direto com `fseek`, `ftell` e `rewind`

Além da leitura sequencial, é possível mover a posição atual dentro do arquivo:

```
int fseek(FILE *arq, long deslocamento, int origem);
long ftell(FILE *arq);
void rewind(FILE *arq);
```

As origens são `SEEK_SET` (início), `SEEK_CUR` (posição atual) e `SEEK_END` (fim).

Para descobrir o tamanho de um arquivo:

```
fseek(arq, 0, SEEK_END);
long tamanho = ftell(arq);
rewind(arq);
```

Para ler um registro específico de um arquivo binário:

```
long indice = 1;
if (fseek(arq, indice * (long)sizeof(Contato), SEEK_SET) == 0) {
    if (fread(&c, sizeof c, 1, arq) == 1) {
        printf("%s\n", c.nome);
    }
}
```

O acesso direto funciona melhor quando todos os registros têm o mesmo tamanho. Ele é a base de muitas implementações simples de cadastros em arquivo.

3. Boas práticas e erros comuns



Strings

- reserve espaço para o terminador `'\0'`;
- prefira `fgets` a `gets`;
- limite a largura em `scanf("%s")`, como em `%99s`;
- não compare strings com `==`; use `strcmp`;
- lembre que `strlen` não conta o terminador e que `sizeof` de um ponteiro não mede a string;
- verifique o espaço disponível antes de usar `strcpy` e `strcat`;
- considere `strncpy` e `strncat`, garantindo o terminador manualmente;
- saiba que `strtok` altera a string original;
- converta para `unsigned char` antes de chamar funções de `ctype.h`.

Arquivos

- teste o retorno de `fopen` antes de usar o ponteiro;
- feche todos os arquivos com `fclose`;
- use `"w"` com cuidado, pois ele apaga o conteúdo anterior;
- use `"a"` quando a intenção é acrescentar dados;
- não controle a leitura com `feof`; use o retorno das funções de leitura;
- guarde o retorno de `fgetc` em `int`;
- use `ferror` para distinguir erro de fim de arquivo;
- não grave ponteiros em arquivos binários;
- verifique se o retorno de `fread` e `fwrite` corresponde ao esperado.

Depuração

- se o arquivo não abrir, confira o caminho e use `perror`;
- se a leitura parar antes do fim, verifique o modo de abertura e o tamanho dos buffers;
- se aparecerem caracteres estranhos, confira o terminador `'\0'` e a codificação;
- se o programa gravar dados incorretos, revise os argumentos de `sizeof` e o formato usado.

